Lec 8

Thursday, September 17, 2026 11:06 AM

CS 6515: Graduate Algorithms

Lecture 8: Maximum Flow and Minimum Cut

Instructors: Sahil Singla and Gerandy Brito

Fall 2026

From matching to flow. Last lecture, an augmenting path improved a matching by undoing some old choices and making new ones. Maximum flow generalizes this idea: we route quantities through a directed network, and reverse residual edges let us revise earlier routing decisions. We will again use BFS to find paths, then use a cut to certify that no further improvement is possible.

1 Flow networks

A flow network is a directed graph $G = (V, E)$ with a nonnegative integer capacity c_e on every edge e, a source $s \in V$, and a sink $t \in V$, with $s \neq t$. Write $n = |V|$ and $m = |E|$. Capacities bound how much flow an edge can carry; they are not path lengths. We assume integral capacities for the running-time and termination analysis below; the definition of a flow still permits real values.

Definition (feasible flow). A flow assigns a nonnegative number f_e to every edge and satisfies:

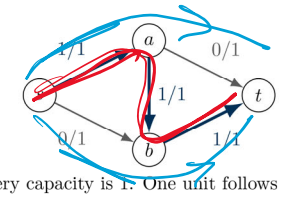
1. **capacity constraints:** $0 \leq f_e \leq c_e$ for all $e \in E$;
2. **flow conservation:** for every $v \in V \setminus \{s, t\}$,

$$\sum_{(u,v)\in E} f_{uv} = \sum_{(v,w)\in E} f_{vw}.$$

The value of a flow is its net flow out of the source:

$$|f| = \sum_{(s,v)\in E} f_{sv} - \sum_{(v,s)\in E} f_{vs}.$$

The maximum-flow problem asks for a feasible flow of maximum value.

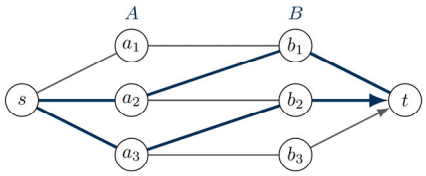


edge labels: f_e/c_e
blue: one unit of flow
$|f| = 1$

Running example. Every capacity is 1. One unit follows s, a, b, t; conservation holds at both a and b. Can we send a second unit without revising this route?

1.1 Bipartite matching as maximum flow

For a bipartite graph with sides A and B, direct every original edge from A to B and give it capacity 1. Add a source edge (s, a) of capacity 1 for each $a \in A$, and a sink edge (b, t) of capacity 1 for each $b \in B$. Every integral flow unit selects one edge between A and B, and capacity 1 ensures no vertex is selected twice. Thus an integral maximum flow corresponds to a maximum matching.



All edges have capacity 1. The dark edges carry one unit each, encoding the Lecture 7 matching $\{(a_2, b_1), (a_3, b_2)\}$ as a flow of value 2.

Why integrality matters. For an integral flow, conservation and the source/sink capacities ensure that the middle edges with flow 1 form a matching. A fractional flow need not select a matching this way. We will show that integer capacities guarantee an *integral optimal flow*, completing the reduction.

Exercise. Prove both directions of the correspondence: turn a matching into a flow, and turn an integral flow into a matching of the same value.

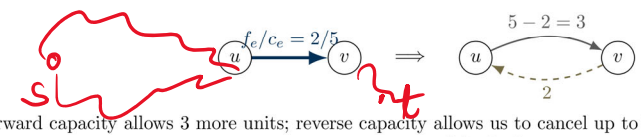
2 Residual graphs

A current flow may use an edge that we later regret. A residual graph records both unused capacity and our ability to cancel previously sent flow.

Definition (residual graph). Given a feasible flow f, the residual graph G_f has vertex set V. For each original edge $e = (u, v)$:

- it has a forward residual edge (u, v) of capacity $c_e - f_e$;
- it has a reverse residual edge (v, u) of capacity f_e.

Only edges of positive residual capacity need to be stored. If residual edges with the same ordered endpoints arise (for example, from original edges in both directions), keep them as separate labeled edges, each linked to its original edge. This makes every update unambiguous.



Forward capacity allows 3 more units; reverse capacity allows us to cancel up to 2 units. A reverse edge does not create an original edge.

An s-t path in G_f is an *augmenting path*. Its bottleneck is

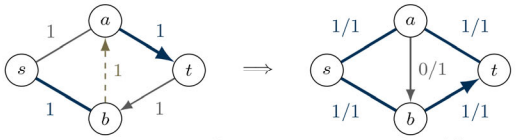
$$\Delta = \min\{c_f(e) : e \text{ lies on the path}\}.$$

Augmenting by Δ increases flow on an original edge traversed forward and decreases flow on an original edge traversed backward. The result remains feasible and its value increases by Δ.

Why feasibility is preserved. The bottleneck prevents any original flow from exceeding capacity or falling below zero. Each residual edge changes the net outgoing flow at its tail by $+\Delta$ and at its

original edge traversed backward. The result remains feasible and its value increases by Δ.

Why feasibility is preserved. The bottleneck prevents any original flow from exceeding capacity or falling below zero. Each residual edge changes the net outgoing flow at its tail by $+\Delta$ and at its head by $-\Delta$, whether it adds or cancels flow. These changes cancel at every internal path vertex. Only the source and sink have a net change.



residual graph of the value-1 flow updated flow, $|f| = 2$

In the running example, augment along s, b, a, t with bottleneck 1. The dashed reverse edge $b \to a$ cancels the old unit on $a \to b$. The updated flow uses the two routes s, a, t and s, b, t.

Connection to matching. In the unit-capacity matching network, a used middle edge has only a reverse residual edge, and an unused one has only a forward edge. These are exactly the orientations from Lecture 7. Flow augmentation generalizes flipping matched and unmatched edges.

Exercise. After the first route s, a, b, t in the running example, show that a search using only unused forward capacity would get stuck. Which reverse edge repairs this decision, and why does using it preserve conservation?

3 The Ford-Fulkerson method

1. Start with the zero flow.
2. Construct the residual graph G_f.
3. Find any s-t path P in G_f. If none exists, stop.
4. Augment by the bottleneck capacity of P and return to step 2.

If all capacities are integers, all flow and residual values remain integral. Every augmentation then increases $|f|$ by at least one. A graph search takes $\mathcal{O}(m+n)$ time, so a simple bound is

$$\mathcal{O}\big((m+n)(1+|f^*|)\big),$$

where $|f^*|$ is the maximum-flow value. This is pseudo-polynomial because $|f^*|$ may be exponentially large in the number of bits used to encode the capacities. The extra 1 accounts for initialization and the final unsuccessful search. This is the same distinction we encountered for value-indexed knapsack: dependence on a numerical value can be much larger than dependence on its bit length. Unlike the FPTAS preview, here the goal remains an exact optimum.

Choosing shortest augmenting paths by BFS gives the Edmonds-Karp algorithm, whose running time is polynomial in n and m, independent of capacity magnitudes. Here "shortest" means *fewest residual edges*, so ordinary BFS suffices; capacities are not lengths. We will not prove its running-time bound here. With arbitrary real capacities, unrestricted path choices in Ford-Fulkerson need not terminate; the BFS variant avoids this issue.

Theorem (augmenting-path optimality). If the residual graph G_f contains no s-t path, then f is a maximum flow.

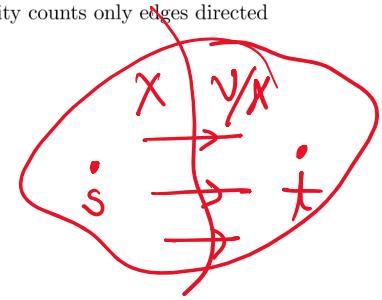
To prove this statement and obtain an optimality certificate, we introduce cuts.

Proof plan. An augmenting path certifies that the current flow is improvable. If search fails, we want the vertices it reached to provide the opposite certificate: a cut whose capacity equals the current flow value. First we show that every cut gives an upper bound on every flow.

4 Minimum s-t cut

An s-t cut is a partition $(X, V \setminus X)$ with $s \in X$ and $t \notin X$. Its capacity counts only edges directed out of X:

$$c(X, V \setminus X) = \sum_{\substack{(u,v)\in E \\ u\in X,\ v\notin X}} c_{uv}.$$



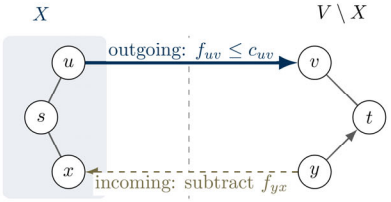
Observation. For every feasible flow f and every s-t cut,

$$|f| \le c(X, V \setminus X).$$

Thus maximum-flow value is at most minimum-cut capacity.

Reason. Summing flow conservation over vertices of $X \setminus \{s\}$ cancels all flow on edges internal to X. Hence $|f|$ equals flow leaving X minus flow entering X, which is at most the capacity of edges leaving X.

Thm: $|f^*| = \min_X c(X, V\setminus X)$



A schematic directed cut. Only outgoing capacities count in its capacity; incoming flow is subtracted when computing net flow across it.

Compare with the MST cut property. For an MST we used the *lightest undirected crossing edge* to justify one greedy choice. For flow we sum *all outgoing capacities* to bound the entire objective. The partition idea is shared, but its role differs.

Theorem (max-flow min-cut). The maximum value of an s-t flow equals the minimum capacity of an s-t cut.

Proof. For the integer capacities assumed here, Ford-Fulkerson terminates by the preceding analysis. Let f be its final flow and let X be the set of vertices reachable from s in G_f. Then $s \in X$ and $t \notin X$.

Consider an original edge (u, v) leaving X. Its forward residual capacity must be zero; otherwise v would also be reachable. Therefore

$$f_{uv} = c_{uv}.$$

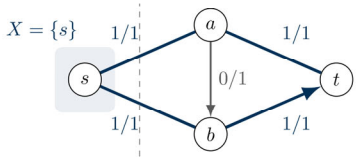
Now consider an original edge (v, u) entering X, where $v \notin X$ and $u \in X$. If $f_{vu} > 0$, its reverse residual edge (u, v) would have positive capacity, again making v reachable. Therefore

$$f_{vu} = 0.$$

The net flow across the cut is consequently

$$|f| = \sum_{\substack{(u,v)\in E \\ u\in X,\ v\notin X}} f_{uv} - \sum_{\substack{(v,u)\in E \\ v\notin X,\ u\in X}} f_{vu} = c(X, V \setminus X).$$

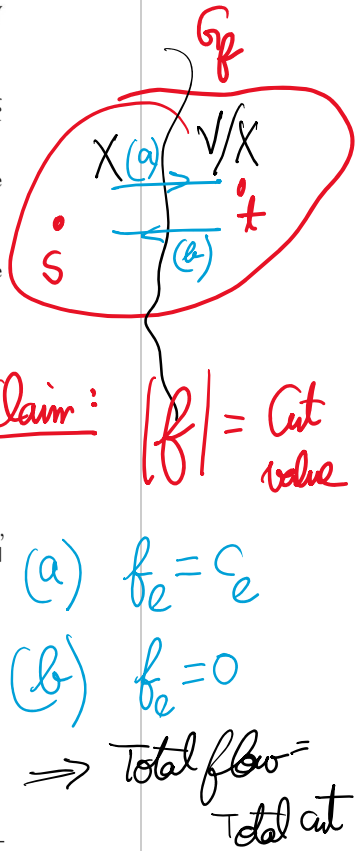
Weak duality gave $|f'| \le c(X, V \setminus X)$ for every feasible flow f'. Our flow reaches this upper bound, so it is maximum and the displayed cut is minimum. □



The final flow in our running example. Both edges leaving s are saturated, so only s is reachable in the residual graph. This cut certifies optimality; the picture shows original edges labeled by flow/capacity.

Returning to bipartite matching. Starting from zero with integer capacities, every augmentation is integral. Our algorithm therefore produces an integral maximum flow. Applied to the unit-capacity matching network, its flow-1 middle edges give a maximum matching, as promised earlier.

Exercise. Explain how the proof gives a short, checkable certificate that a proposed flow is maximum. What data would you hand to someone who did not run the algorithm?



5 Part I synthesis

The first unit has developed three broad algorithmic lenses:

- **divide and conquer:** split, recurse, and combine;
- **dynamic programming:** define overlapping states and reuse their values;
- **graph algorithms:** exploit reachability, cuts, paths, and local exchange structure.

For every algorithm, we paired a construction with a proof invariant and an explicit running-time analysis.

The graph algorithms illustrate two ways to justify progress. Prim makes irrevocable choices using an exchange argument. Matching and flow revise earlier choices along augmenting paths, and a failed search certifies optimality. Finding a solution and explaining why it is optimal are both parts of algorithm design.

Looking ahead to linear programming. The flow constraints are linear equalities and inequalities, and the flow value is a linear objective. The statement that every feasible flow is bounded by every cut is an example of *weak duality*; equality at the optimum previews the duality theory we will develop in Part II.

Exercise. For each of Prim, bipartite matching, and maximum flow, state the maintained invariant, the operation that makes progress, and the argument or certificate used to prove optimality. Which algorithms can undo an earlier choice?